

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA11

COURSE OUTCOME COVERED

***CO1:** Apply object-oriented concepts and software development life cycle models to design structured and modular software systems. (BL2, BL3)*

Assessment Tool Weightage: 60%

Dr Ramamoorthy

QUESTIONS: 12

Total Marks:60

Annexure A

CONCEPT MAPPING

Code of Conduct:

I DARSHINIE KARANI V M(192520033) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

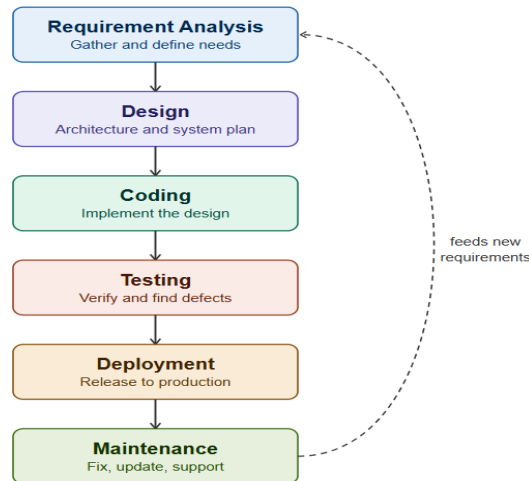
Annexure A

CONCEPT MAPPING

1. Construct a Concept Map

Given:

Requirement Analysis → **Design** → Coding → Testing → **Deployment** → Maintenance



Explanation:

- Design converts requirements into system architecture.
- Coding translates the design into actual program code.
- Testing verifies the software against requirements and finds defects.
- Deployment releases the software to end users.
- Maintenance handles bug fixes, updates, and enhancements after release.

Analysis:

- Ensures systematic software development.
- Reduces project risks.
- Improves software quality and maintainability.
- Provides clear milestones for tracking project progress.

(5 Marks)

2. Construct a Concept Map

Given:

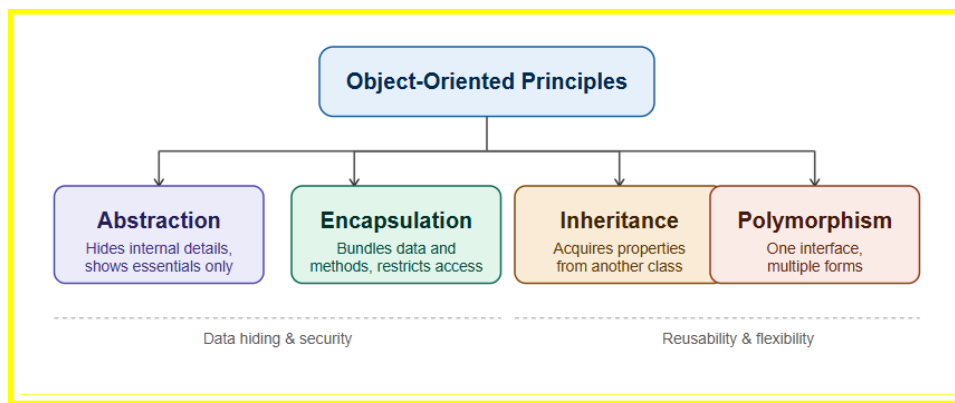
Object-Oriented Principles

→ Abstraction

→ Encapsulation

→ Inheritance

→ Polymorphism



Explanation:

- Abstraction hides complex implementation and exposes only necessary functionality.
- Encapsulation protects data from unauthorized access.
- Inheritance enables a class to reuse and extend properties of a parent class.
- Polymorphism allows one interface to support multiple behaviors.

Analysis:

- Improves security and flexibility.
- Enhances code reusability.
- Supports efficient software design.
- Simplifies debugging by isolating functionality within objects.
- Promotes modularity, allowing independent development of components.
- Makes software easier to scale and extend with new features.
- Encourages real-world modeling, since classes and objects map naturally to real entities

(5 Marks)

3. Construct a Concept Map

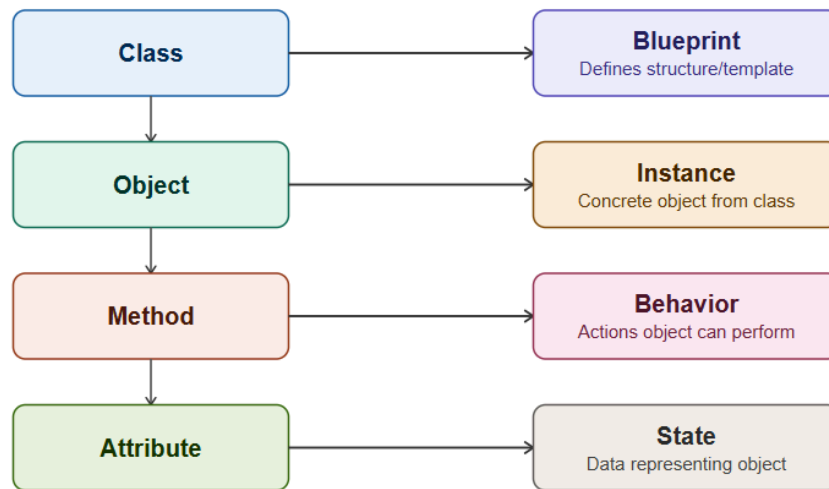
Given:

Class → Blueprint

Object → Instance

Method → Behavior

Attribute → State



Explanation:

- A class acts as the blueprint from which objects are created.
- An object is an instance created from a class.
- Methods define the behavior of objects.
- Attributes represent the state of objects.

Analysis:

- Helps model real-world entities.
- Defines object characteristics and actions.
- Improves system organization.
- Separates data (state) from functionality (behavior), making code cleaner.
- Enables multiple objects to be created from a single class without duplicating code.
- Supports maintainability, since changes to the blueprint automatically apply to future instances.
- Forms the conceptual basis for advanced OOP features like inheritance and polymorphism.

(5 Marks)

4. A Concept Map Shows

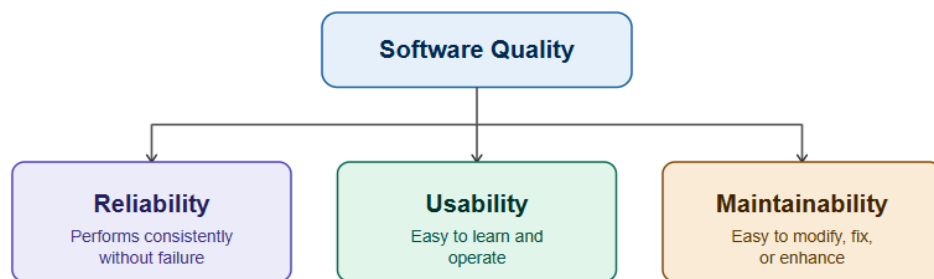
Given:

Software Quality

→ Reliability

→ **Usability**

→ Maintainability



Explanation:

- Reliable software performs consistently without failure.
- Usability measures how easily users interact with software.
- Maintainable software is easy to update.
- These three attributes together are core indicators of overall software quality.
- Poor performance in any one area can undermine the overall success of the software.

Analysis:

- Improves user satisfaction.
- Enhances product acceptance.
- Contributes to software success.
- Reduces long-term maintenance and support costs.
- Builds user trust through consistent, error-free performance.
- Increases adoption rate by making the software accessible to a wider range of users.
- Supports easier scaling and future feature additions due to maintainable code structure.

(5 Marks)

5. UML Concept Map

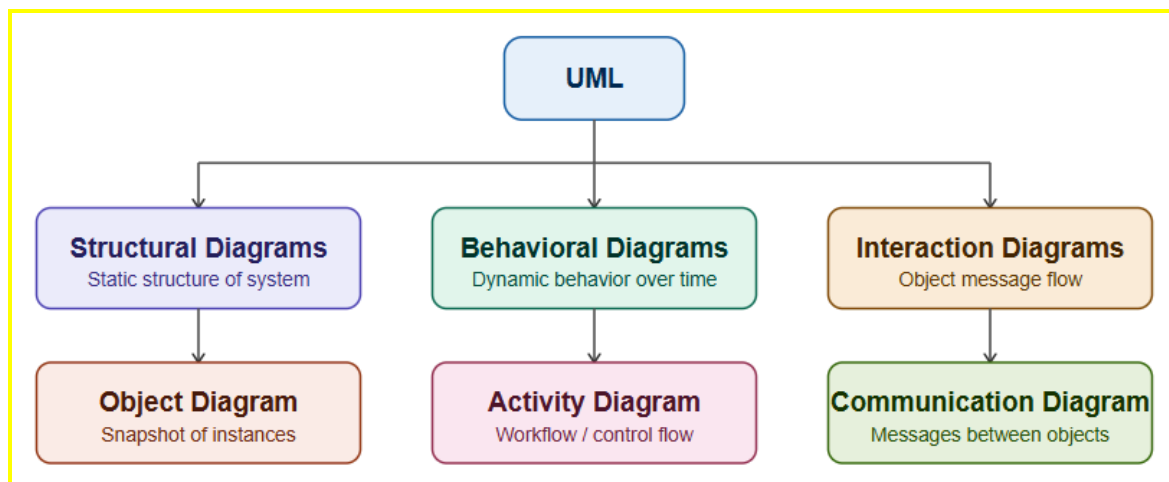
Given:

UML

→ Structural Diagrams → Object Diagram

→ Behavioral Diagrams → Activity Diagram

→ Interaction Diagrams → Communication Diagram



Explanation:

- Structural diagrams show the static structure of a system, including classes, objects, and their relationships.
- Object diagrams represent object instances and relationships at a specific point in time.
- Behavioral diagrams depict how the system behaves and changes over time.
- Activity diagrams model the workflow or control flow of a process.
- Interaction diagrams focus on how objects communicate with each other.
- Communication diagrams show message exchange among objects.

Analysis:

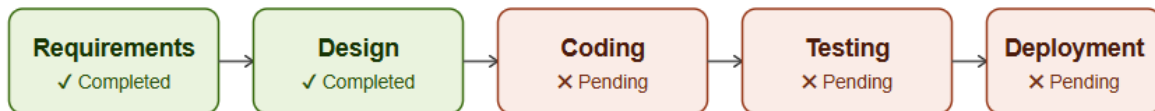
- Improves system visualization.
- Supports object-oriented analysis.
- Facilitates stakeholder communication.
- Helps identify design flaws early by making system structure and behavior explicit.
- Bridges the gap between requirements and implementation through visual modeling.
- Object diagrams validate class diagrams by showing real instance data.
- Communication diagrams complement sequence diagrams by emphasizing object relationships over strict time order.

- Standardized UML notation improves collaboration across development teams.

(5 Marks)

6. Given:

Requirements ✓ → Design ✓ → Coding ✗ → Testing ✗ → Deployment ✗



Explanation:

- Failure occurs during the Coding phase.
- Program logic contains implementation errors.
- Requirements and Design phases were completed successfully, so the foundation for coding was sound.
- The failure indicates a gap between the design specification and actual code implementation.

Analysis:

- Errors affect subsequent phases.
- Testing cannot proceed effectively.
- Software quality decreases.
- Delays cascade through the remaining phases, pushing back the overall project timeline.
- Increases the cost of fixing defects, since bugs caught later are more expensive to resolve than those caught early.
- Reduces team confidence and stakeholder trust in the project's progress.
- Highlights the need for stronger coding practices and more frequent code reviews going forward.

Solution:

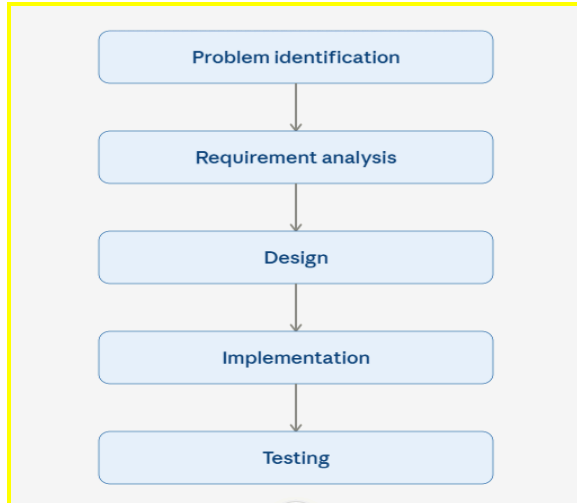
- Review source code.
- Apply coding standards.
- Conduct code inspections before testing.

(5 Marks)

7. Construct a Concept Map

Given:

Problem Identification → Requirement analysis → Design → Implementation → Testing



Explanation:

- Requirement Analysis also involves feasibility study to check if the project is technically and financially viable.
- Design can be split into high-level design (overall architecture) and low-level design (detailed module logic).
- Implementation follows coding standards and best practices to ensure readability and maintainability.
- Testing includes multiple levels such as unit testing, integration testing, and system testing.
- Problem Identification often involves stakeholder interviews and market study to validate the need for the solution.

Analysis:

- Increases customer satisfaction by aligning the final product with actual user needs.
- Minimizes rework by catching design flaws before coding begins.
- Improves system scalability and reliability through structured design practices.
- Supports risk management by identifying potential technical challenges early.

(5 Marks)

8. Construct a Concept Map

Given:

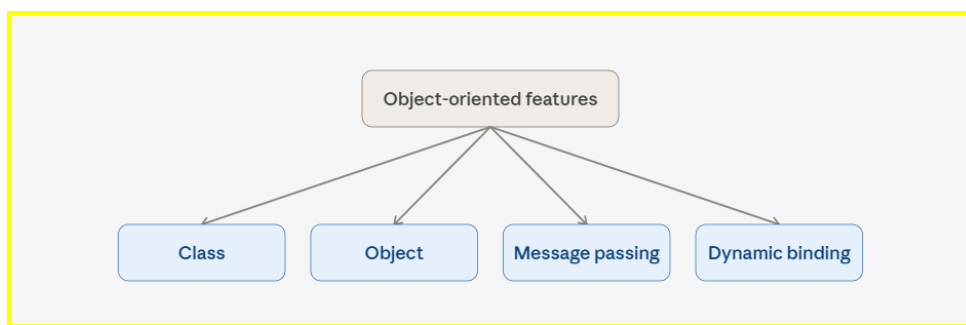
Object-Oriented Features

→ Class

→ **Object**

→ Message Passing

→ **Dynamic Binding**



Explanation:

- A class acts as a blueprint that defines the attributes and behaviors shared by all its objects.
- Message passing enables objects to communicate by invoking each other's methods.
- Message passing allows objects to interact without exposing their internal implementation details.
- Dynamic binding is also known as late binding, since the method to execute is decided during program execution rather than at compile time.

Analysis:

- Encourages modularity by allowing objects to interact through well-defined interfaces.
- Reduces coupling between components since objects communicate via messages instead of direct access.
- Simplifies debugging and testing since each class encapsulates its own logic.
- Enhances code reusability by allowing new classes to reuse existing class structures.
- Facilitates extensibility, as new object types can be added without modifying existing code.

- Improves system responsiveness by allowing behavior to adapt based on the actual object type at runtime.

(5 Marks)

9. Construct a Concept Map

Given:

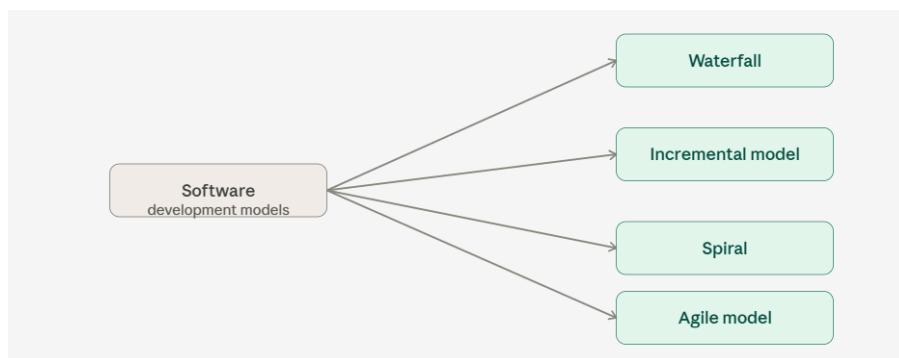
Software Development Models

→ Waterfall

→ Incremental model

→ Spiral

→ Agile Model



Explanation:

- Waterfall follows a strict sequential process, where each phase must be completed before the next begins.
- Spiral model combines iterative development with systematic risk analysis at each cycle.
- Waterfall works best for projects with clear, stable requirements known in advance.
- Spiral is well suited for large, complex, and high-risk projects.
- Agile relies on short development cycles called sprints, with regular reviews and adjustments.

Analysis:

- Increases transparency, since stakeholders see working software early and often.
- Allows early identification of design or requirement flaws before they become costly.
- Supports better resource planning by breaking large projects into manageable phases.
- Encourages team collaboration through frequent communication and feedback loops.
- Improves final product quality by allowing continuous testing and refinement.
- Provides flexibility to accommodate changing requirements during development

(5 Marks)

10. Construct a Concept Map

Given:

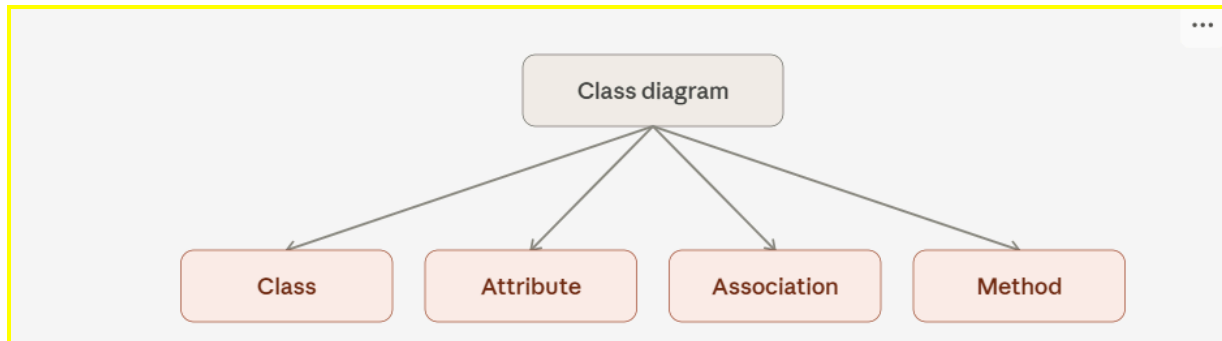
Class Diagram

→ Class

→ **Attribute**

→ Association

→ **Method**



Explanation:

- A class in a class diagram represents a template with a name, attributes, and methods, shown as a rectangle divided into three sections.
- Association represents relationships between classes, showing how objects of one class are connected to objects of another.
- Attributes are typically shown with their data type, such as name: String or age: int.
- Methods are shown with their parameters and return type, indicating what operations the class can perform.

Analysis:

- Helps visualize the static structure of a system before implementation begins.
- Makes it easier to identify relationships and dependencies between different classes.
- Serves as a communication tool between developers, designers, and stakeholders.
- Reduces ambiguity in system design by clearly defining class responsibilities.
- Assists in identifying reusable components across different parts of the system.
- Provides a foundation for generating code structure in object-oriented programming languages

(5 Marks)

11. Construct a Concept Map

Given:

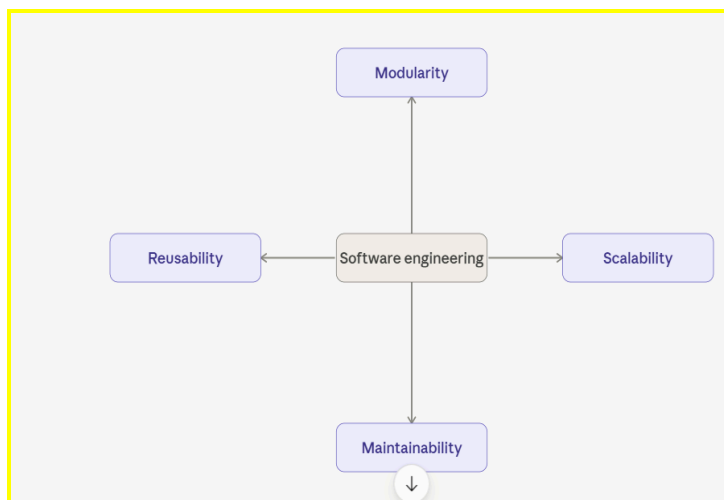
Software Engineering Principles

→ Modularity

→ Maintainability

→ Reusability

→ Scalability



Explanation:

- Modularity breaks a system into independent, self-contained components that can be developed and tested separately.

- Reusability allows existing code or components to be used again in different projects, reducing development time.
- Modular systems make it easier to isolate and fix bugs without affecting other parts of the software.
- Scalability can be achieved either by adding more resources to a single system or by distributing load across multiple systems.

Analysis:

- Lowers overall development cost by reusing tested and proven components.
- Enables parallel development, as different modules can be worked on simultaneously by different teams.
- Increases system reliability, since well-defined modules are easier to validate and debug.
- Enhances flexibility to integrate new technologies or features without major redesign.
- Improves fault isolation, so a failure in one module has minimal impact on the overall system.
- Supports better resource utilization as the system grows in size and complexity

(5 Marks)

12. Given:

Requirement Analysis ✓ → System Design ✓ → Coding ✓ → Integration ✗ → Deployment ✗

Explanation:

- Integration failures often arise from mismatched data formats or incorrect parameter passing between modules.
- Interface mismatches, such as differing API contracts, are a common root cause of integration failure.
- Dependency conflicts between modules developed by different teams can also trigger integration issues.

Analysis:

- Reduces overall system reliability if integration issues go undetected.
- Increases project cost due to rework and repeated testing cycles.
- Affects team coordination, since multiple teams may need to revisit their modules.
- Can lead to cascading failures if one faulty module affects dependent modules downstream.

Solution:

- Use stub and driver modules to test incomplete interfaces during early integration.
- Adopt continuous integration practices to detect issues as soon as new code is merged.
- Maintain clear and updated interface documentation to prevent miscommunication between teams.
- Conduct incremental integration rather than integrating all modules at once, to isolate failures faster.

(5 Marks)

CONCEPT MAPPING

RUBRICS FOR CALCULATION

CONCEPT MAPPING

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand